



CHAPTER – 5

Subroutines and Control Abstraction

Subroutines

- **Abstraction:** a process of **associating a name with** a potentially complicated program fragment.
 - ❑ **Control abstraction**, the principal purpose is to perform a well defined **operation**
 - ❑ **Data abstraction**, the principal purpose is to **represent information**.
- **Subroutines:-** are the principal mechanism for **control abstraction** in most programming languages(give a name to a task)
- The *subroutine call is an expression(operation)*
 - ❑ Subroutines take arguments (in the formal parameters)
 - ❑ Values are placed into variables (actual parameters/arguments), and
 - ❑ A value is (usually) returned

Review of Stack Layout

- Each routine, as it is called, is given a new *stack frame*, or *activation record*, at the *top of the stack*.
- When a subroutine returns, its frame is popped from the stack.

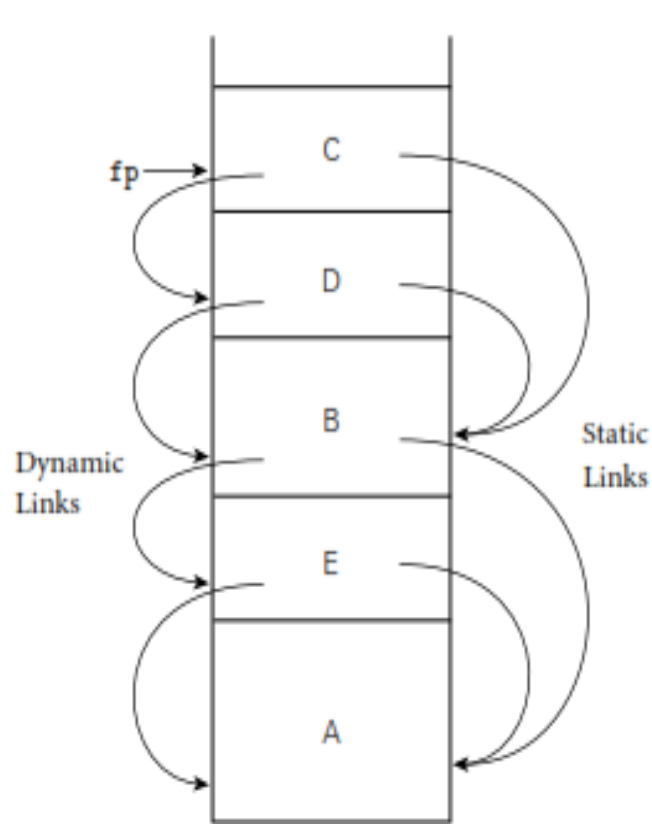
Contents of a stack frame:

- Bookkeeping (return address and saved registers)
- arguments and returns
- local variables
- Temporaries

Stack pointer: The address of either:

- last used location at the top of the stack
- First unused location

- **Frame pointer:** The address within the frame.
 - ❑ Objects in the frame are accessed via offset(displacement address) from the frame pointer
 - ❑ **Variable size objects** are placed in a variable-size area at the top of the frame.
 - ❑ Its **address and dope vector** (descriptor) are stored in the fixed-size portion of the frame,
- If no variable-size objects, then everything has fixed offset from stack pointer
 - ❑ In this case, frame pointer isn't needed at all
 - ❑ Frees up a register for use
- If the size of an argument is not known at compile time,
 - ❑ the argument may be placed in a variable-size portion of the frame below the other arguments,



Static and dynamic links maintained on stack:

- *Static store context for scope purposes/reference to the frame*
- *Dynamic store saved value of the frame pointer, so that it returns to correct calling method*

Fig. Example of subroutine nesting: Within B, C, and D, all five routines are visible. Within A and E, routines A, B, and E are visible, but C and D are not. Given the calling sequence A, E, B, D, C, in that order, frames will be allocated on the stack as shown at right, with the indicated static and dynamic links.

Calling Sequence

- Maintenance of the subroutine call stack is the responsibility of the calling Sequence.
 - The code executed by the caller immediately before and after a subroutine
 - *Prologue* (code executed at the beginning) and
 - *Epilogue* (code executed at the end) of the subroutine itself
- Tasks that must be accomplished **on the way into** a subroutine include:
 - ☐ Passing parameters,
 - ☐ Saving the return address,
 - ☐ Changing the program counter,
 - ☐ Changing the stack pointer to allocate space,
 - ☐ Saving registers (including the frame pointer) that contain important values and that may be overwritten by the callee,

Calling Sequence..

- Tasks that must be accomplished **on the way out** include:-
 - ❑ passing return parameters or function values,
 - ❑ executing finalization code for any local objects that require it,
 - ❑ deallocating the stack frame (restoring the stack pointer),
 - ❑ restoring other saved registers (including the frame pointer), and
 - ❑ restoring the program counter.
- Some of these tasks (e.g., passing parameters) must be performed by the caller, because they differ from call to call.
- Most of the tasks, however, can be performed either by the caller or the callee.

Calling Sequence..

- In general, space is saved if work is mostly done by callee:
 - ❑ Anything done in callee appears only once in the target program but,
 - ❑ Anything done by caller has to appear before and after every call in the final compiled code
- The trickiest division-of-labor issue pertains to **saving registers**
 - The ideal approach is to **save** precisely those **registers** that are both in use **in the caller** and needed for other purposes **in the callee**.
 - This is almost impossible to do perfectly b/c of separate compilation
- A simpler solution is for the caller to save all registers that are in use, or for the callee to save all registers that it will be overwrite.

Calling Sequence..

Calling sequence conventions for many processors (MIPS and x86 among others)

- Two sets: caller responsibility and callee responsibility.
 - Callee can assume that there is nothing of value in any of the caller saved register set.
 - Caller can assume no callee will destroy contents in any of the callee saves set.
- In the interests of code size ,compiler will then use
 - **callee-saves** registers for **local variables** and other long-lived ones if possible &

Calling Sequences: RISC architecture

- It is worth noting something about RISC architecture:
 - Heavily optimized to support simple instructions, so that every operation takes one clock cycle.
 - Advantage is that pipelining is much easier in this setup
 - Each command also requires less transistors of hardware space
- In contrast, CISC architecture:
 - Supports more complex operations: mult as well as add
 - Advantage is that code size is much smaller, so less RAM needed
 - In addition, less complexity in compiler

Calling Sequences (RISC architecture)

- Why RISC (reduced instruction set computer) Matters to us:
 - ❑ In general, RISC architecture forces more work to save registers
- Calling sequence (C on MIPS) operate as follows:
 - **Caller** need to (for C on MIPS) :
 - ❑ Saves into the *“local variable and temporaries”* area any caller-saves registers whose values are still needed.
 - ❑ puts up to **4 small arguments into registers r0-r3**.
 - ❑ puts the rest of the arguments into the argument build area at the top of the current frame
 - ❑ puts return address into register, **jumps to target** address, and (optionally) changes instruction set coding

- In prolog, Callee

- ☐ Subtracts frame size from sp to allocates a frame.
- ☐ saves the old frame pointer into the stack, and assigns it an appropriate new value copies
- ☐ subtracts from **sp to make space for local variables**, temporaries, and arg.
build area at top of stack

- In epilog, Callee

- ☐ puts return value into registers or a reserved location in the stack
- ☐ restores saved registers
- ☐ restores the fp and the sp (copies fp into sp)
- ☐ adds to sp to deallocate frame and jumps back to the return address

Finally, the caller

- ❑ Moves the return value to wherever it is needed
- ❑ Restores caller-saves registers if needed

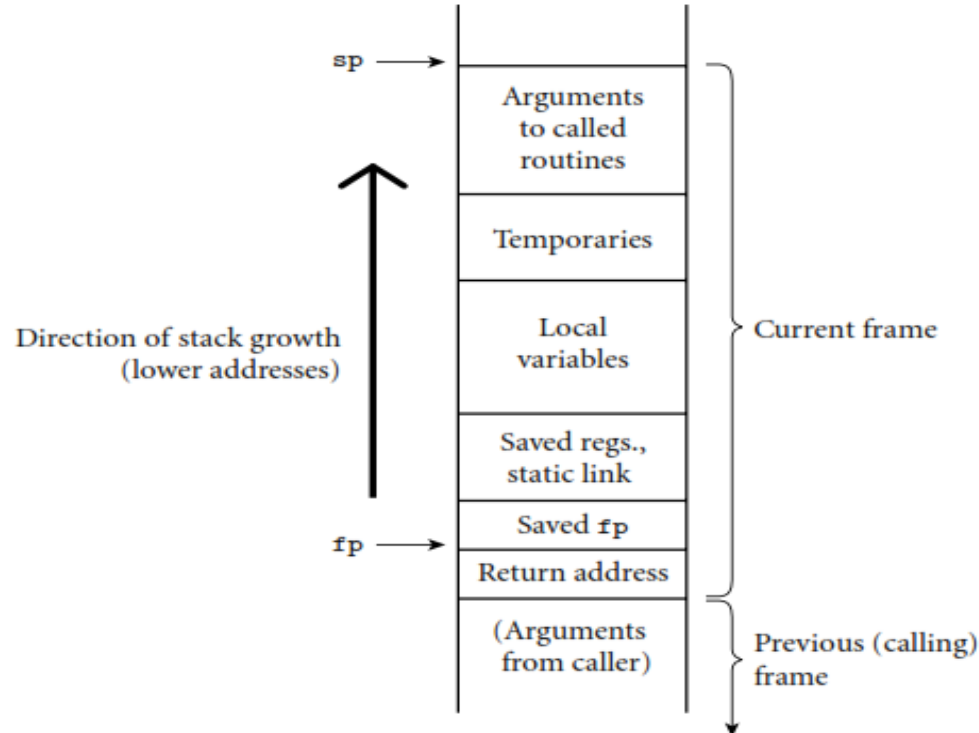


Fig. A typical stack frame. Though we draw it growing upward on the page, the stack actually grows downward toward lower addresses on most machines. **Arguments are accessed at positive offsets from the fp.** **Local variables and temporaries are accessed at negative offsets from the fp.** Arguments to be passed to called routines are assembled at the top of the frame, using positive offsets from the sp.

In-Line Expansion

- In some languages, programmers can actually flag routines that should be expanded inline
 - stack overhead is avoided.

- Example (in C++ or C99):

```
inline int max(int a,int b)  
{ return a > b ? a : b }
```

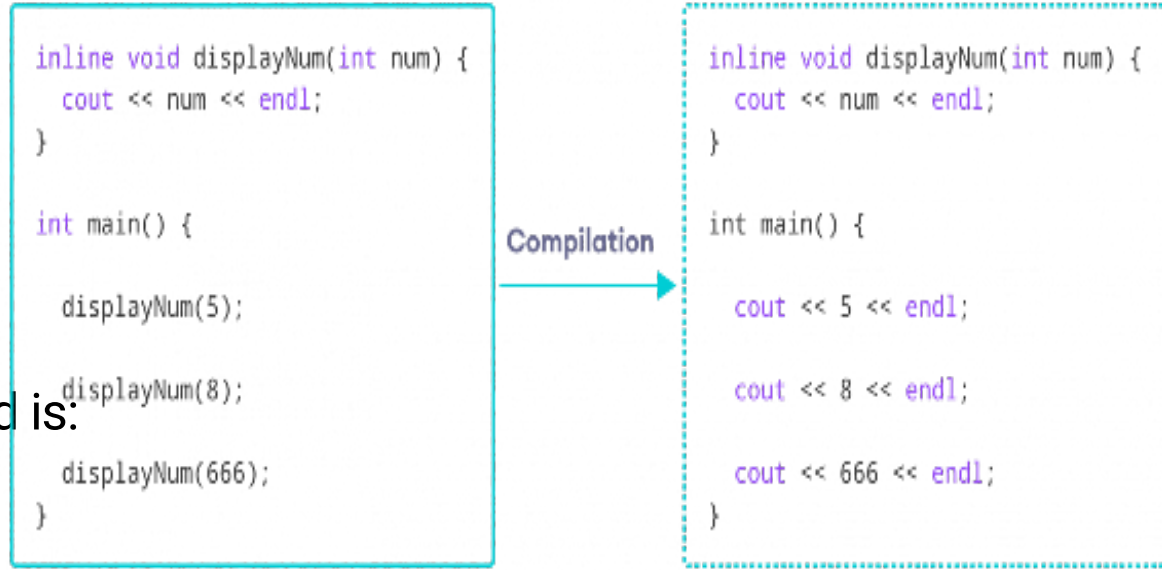
- Ada does something similar, but keyword is:

```
pragma inline(max);
```

- A copy of the called routine becomes a part of the caller

❑ no actual subroutine call occurs.

- In-line expansion avoids a variety of overheads, including space allocation



Parameter Passing

- **Formal parameters:** Parameter names that appear in the declaration of a subroutine
- **Actual parameters:** Variables and expressions that are passed to a subroutine in a particular call.

```
int main()
{
    add(m,n)      }    // Actual parameters
}
int add(int a, int b)    // Formal parameters
{
    return (a+b);
}
```

Parameter Passing

- Parameter passing modes:
 - Call by value: make a copy of the parameter.
 - Call by reference (aliasing): allows the function to change the parameter
 - The two names are aliases for the same object, and changes made through one will be visible through the other
 - closure/name: reference to a subroutine, together with its referencing environment
 - Call by sharing: requires parameter to be a reference itself.
 - Makes copy of reference that initially refers to the same object. E.g., Python, Java Objects.
- Many languages (e.g., Pascal, Ada, Modula) provide different modes
- Others have a single set of rules (e.g. C, Fortran, ML, Lisp)

```
x : integer                -- global
procedure foo(y : integer)
    y := 3
    print x
...
x := 2
foo(x)
print x
```

- If x is passed to foo by value, then the assignment inside foo has no visible effect , y is private to the subroutine and the program prints 2 twice.
- If x is passed to foo by reference, then the assignment inside foo changes x to y and the program prints 3 twice.

What will be the output if passed by value and passed by reference?

Call by value/result

- A mode first introduced in Algol W.
- Like call by value, call by value/result copies the actual parameter into the formal parameter at the beginning of subroutine execution.
- Unlike call by value, it also copies the formal parameter back into the actual parameter when the subroutine returns.
 - I.e. Like call-by-reference that allows the called routine to modify the actual parameter
- Call by Value/result would copy x into y at the beginning of foo, and y into x at the end of foo

```
x : integer          -- global
procedure foo(y : integer)
  y := 3
  print x
...
x := 2
foo(x)
print x
```

The program would print ?

C/C++: functions

- Parameters passed by value (C)
- Parameters passed by reference can be simulated with pointers (C)

```
void proc(int* x,int y){*x = *x+y }
```

...

```
proc(&a,b);
```

- or directly passed by reference (C++)

```
void proc(int& x, int y) {x = x + y }
```

```
lproc(a,b);
```

Read only parameters : *formal parameters declared read only cannot be changed by called routines.*

- *Similar to c constant and available in Modula -3*

Call-by-value:

- ❑ Can be expensive to implement (e.g., copying large objects).
- ❑ Can't change a parameter, except by returning a new copy.

Call-by-reference:

- ❑ Out-parameters (i.e., the procedure returns values through its parameters).
 - Allows callee to change something and have it persist
 - Saves space/time copying
- ❑ Good: More flexibility.
- ❑ Bad: Can be confusing when arguments change.

Does lead to some confusion with novices:

- ❑ Pragmatic issue: Does the implementation pass by value or reference?
- ❑ Semantic issues: Is the callee allowed to change the variable, & do these changes persist?

Call-by-sharing:

- Pass the reference itself, and let the actual and formal parameters refer to the same object.
 - Clu , LISP calls this mode *call-by-sharing*.
- If we modify the object to which the formal parameter refers, the program will be able to see those changes through the actual parameter after the subroutine returns.
- Some languages mix this up this a hybrid model of variables:
 - Java uses call-by-value for variables of built-in type (all of which are values), and
 - call-by-sharing for variables of user-defined class types (all of which are references)

Ada provides three parameter-passing modes:

- ❑ In parameter: pass information from the caller to the callee; they can be read by the callee but not written.(i.e callee reads only)
- ❑ Out : callee writes & can then read (formal not initialized); actual values is modified
- ❑ In out: callee reads and writes; actual modified

Ada in/out is always implemented as:

- ❑ In is call-by-value
- ❑ In out is call-by-value/result
 - ❑ passed as an address/ reference will affect the actual parameter immediately ,for structured objects
 - ❑ passed as a value will not affect the actual parameter until the subroutine returns, for scalars.

- ❑ out is call-by-result (the value of the formal parameter is copied into the actual parameter

Closures as Parameters (Subroutines as parameters)

- **closures: a reference to a subroutine, together with its referencing environment**
 - Goal: whenever that function is called, want access to its referencing environment
- So even if called in a very different context, still get the information you need to run it correctly.
- Ada 83 did not permit subroutines to be passed as parameters.
- Fortran has always allowed subroutines to be passed as parameters
- Subroutines are routinely passed as parameters (and returned as results) in functional languages.
- C and C++ have no need of subroutine closures, because their subroutines do not nest. Simple pointers to subroutines suffice.

Example: (closure)Subroutines as parameters in Pascal

```
1. procedure apply_to_A(function f(n : integer) : integer;  
2.           var A : array [low..high : integer] of integer);  
3. var i : integer;  
4. begin  
5.   for i := low to high do A[i] := f(A[i]);  
6. end;  
   ...  
7.   var k : integer;           (* in nested scope *)  
   ...  
8.   function add_k (m : integer) : integer;  
9.   begin  
10.    add_k := m + k;  
11.  end;  
   ...  
12.  k := 3;  
13.  apply_to_A(add_k, my_array);
```


Call value vs Call by name

Example:

```
begin
integer n;
procedure p(k:integer);
  begin
    print(k);
    n=n+10;
    print(k);
    n:=n+5;
    print(k);
  end;
n:=0;
p(n+1);
end;
```

Output:

Call by value: 1 1 1

Call by name: 1 11 16

Parameter mode	Representative languages	Implementation mechanism	Permissible operations	Change to actual?	Alias?
value	C/C++, Pascal, Java/C# (value types)	value	read, write	no	no
in, const	Ada, C/C++, Modula-3	value or reference	read only	no	maybe
out	Ada	value or reference	write only	yes	maybe
value/result	Algol W	value	read, write	yes	no
var, ref	Fortran, Pascal, C++	reference	read, write	yes	yes
sharing	Lisp/Scheme, ML, Java/C# (reference types)	value or reference	read, write	yes	yes
in out	Ada	value or reference	read, write	yes	maybe
name	Algol 60, Simula	closure (thunk)	read, write	yes	yes
need	Haskell, R	closure (thunk) with memoization	read, write*	yes*	yes*

Fig. Parameter-passing modes. Column 1 indicates common names for modes. Column 2 indicates prominent languages that use the modes, or that introduced them. Column 3 indicates implementation via passing of values, references, or closures. Column 4 indicates whether the callee can read or write the formal parameter. Column 5 indicates whether changes to the formal parameter affect the actual parameter. Column 6 indicates whether changes to the formal or actual parameter, during the execution of the subroutine, may be visible through the other.

Generic Subroutines and Modules

- Structured programming is away to split your code in blocks.
- Subroutines is a sequence of program instruction that performs a specific task, packaged unit.
- Modules help the programmer to split their code
- Generic modules or classes are particularly valuable for creating containers: data abstractions that hold a collection of objects
 - Example: OS needs queues that can hold processes, memory descriptors, file buffers, etc.
- Generic subroutines (methods) are needed in generic modules (classes), and may also be useful in their own right.
- With large programs containing many method subroutines will be needed to

Generic Subroutines and Modules

Subroutine Add example:

string add(string a, string b)

return a+b;

int add(int a, int b)

return a+b;

double add(double a, double b)

return a+b;

Generic subroutine

Public T <T> add(T a, T b)

return a+b;

int c= add(5,7)

double d= add(4.5, 5.5)

String concat=add("gen", "erics")

- Subroutine should be written once and be capable of accepting any argument. This idea is known as generic programming.
- Generic or polymorphic subroutine is one that takes parameter of different types on different activation.

E.g. Java, C++, Ada, clu, Eiffel, modula-3 ,C#..

Generic Subroutines and Modules

Implementation approaches:

- Ada and C++ they are a purely static mechanism: all the work required to create and use multiple instances of the generic code takes place at compile time.
- The compiler creates a *separate copy of the code for every instance*.
- Java 5, in contrast, guarantees that all instances will share the same code at run time,
 - If T is a generic type parameter in Java, objects of class T are treated as instances of the standard base Object that Java supports

A generic add method in C++ & Java:

```
template <typename T> T add(T a, Tb)  
{  
    return a+b;  
}
```

C++

To call the method:
add(5,6);
add(5.6, 7,8);

```
public <T> T add(T a, Tb)  
{  
    return a+b;  
}
```

Java

Exception Handling

What is an exception?

- A hardware-detected run-time error or unusual condition detected by software

Examples

- arithmetic overflow
- end-of-file on input
- wrong type for input data
- user-defined conditions, not necessarily errors

What is an exception handler?

- code executed when exception occurs
- may need a different handler for each type of exception

Why design in exception handling facilities?

- allow user to explicitly handle errors in a uniform manner

Exception Handling

- Many more recent languages, including Clu, Ada, Modula-3, Python, PHP, Ruby, C++, Java, C#, and ML, all provide exception-handling facilities in which handlers are lexically bound to blocks of code,

A simple try block in C++

```
try {  
    ...  
    if (something_unexpected)  
        throw my_exception();  
    ...  
    cout << "everything's ok\n";  
    ...  
} catch (my_exception) {  
    cout << "oops\n";  
}
```

In Ada, exception is a built-in type; an exception is simply an object of this type:

`declare empty_queue : exception;`

In Modula-3, exceptions are just another object, like constants or variables

`EXCEPTION empty_queue;`

Exception Handling

Exception handlers tend to perform three kinds of operations:

1. A handler will *compensate for the exception in a way that allows the* program to recover and continue execution
2. Declare a local handler to clean up local resources allocated in the local block and then reraise the exception, so that it will continue to propagate back to a handler that can (hopefully) recover.
3. If recovery is not possible, a handler can at least print a helpful error message before the program terminates.

Coroutines

- Coroutines are execution contexts that exist concurrently, but that execute one at a time, and that transfer control to each other explicitly, by name
 - Because they are concurrent (i.e., simultaneously started but not completed), coroutines cannot share a single stack

As a simple application use coroutines:

coroutine check file system

for all files ...

coroutine update screen

loop

update screen

//write a loop which does screen update and
check in
one block, but this mixes tasks

Coroutines

- Coroutines cannot share a single stack: their subroutine calls and returns, taken as a whole, do not occur in last-in-first-out order.
 - If each coroutine is declared at the outermost level of lexical nesting, then their stacks are entirely disjoint.
- A coroutine is represented by a closure (a code address and a referencing environment),
- Coroutines can be used to implement
 - iterators
 - threads

Coroutines

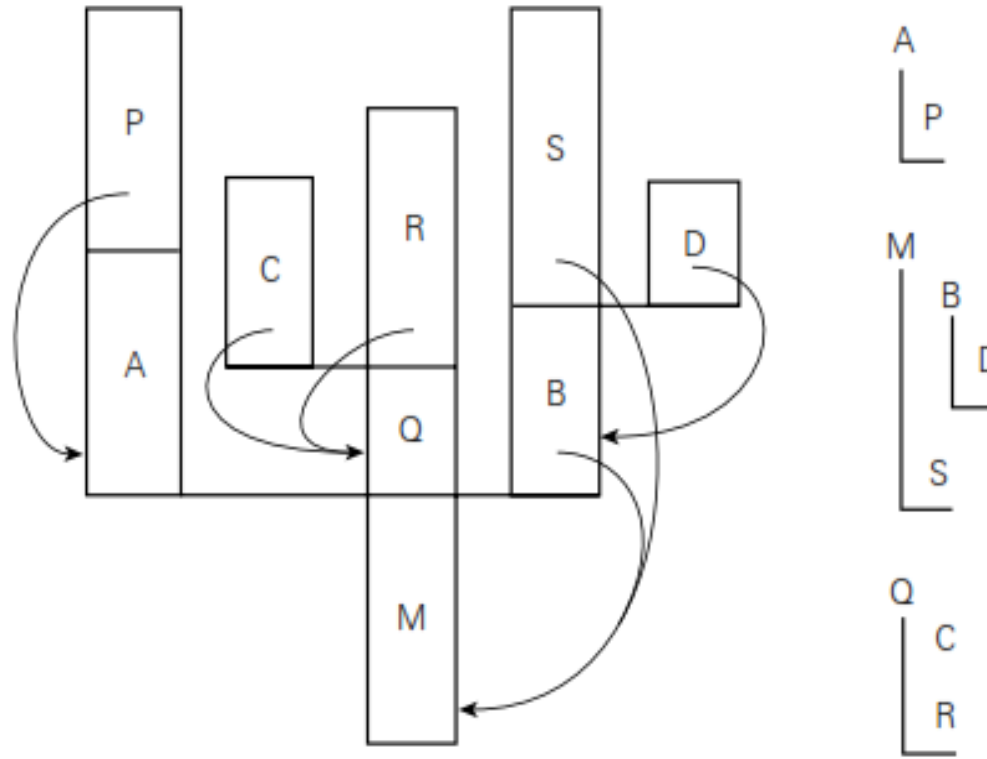


Fig. Coroutines stack allocation. Each branch to the side represents the creation of a coroutine (A, B, C, and D). The static nesting of blocks is shown at right. Static links are shown with arrows. Dynamic links are indicated simply by vertical arrangement: each routine has called the one above it. (Coroutine B, for example, was created by the main program, M. B in turn called subroutine S and created coroutine D.)

End of chapter - 5

```
a:int
Procedure P(b:int)
{
    b=b+1
    b=b+a
}
a=2
P(a)
Print a
```

call by value___
Call by ref___
Call by value-result___

```
a:int
Procedure P(a:int, b:int)
{
    b=a
    b=a
}
a=3
P(a+1,a)
Print a
```

Call by name___
Call by value ___